



Universidad Nacional de La Matanza

ALGORITMOS Y ESTRUCTURAS DE DATOS

Trabajo Práctico Caravana del Desierto

Comision: 03-3600

Integrantes:

Cáceres, Facundo Tomás

Dimattia Damm, Malena

Vazquez, Isaac Benjamin

Documento de decisiones de diseño e implementación

Trabajo Práctico: Caravana del Desierto

Este documento complementa el enunciado y resume las decisiones concretas tomadas durante la implementación.

1. Objetivo del documento

El objetivo de este documento es explicar cómo se implementó el juego Caravana del Desierto, qué estructuras de datos se utilizaron y qué decisiones de diseño tomó el equipo para resolver situaciones que el enunciado dejaba abiertas o que requerían una interpretación de implementación.

2. TDA utilizados:

3. Lista doblemente enlazada circular

El tablero se implementa mediante una lista doblemente enlazada circular. Esta estructura no había sido desarrollada previamente como una biblioteca cerrada por la cátedra, por lo que el equipo consultó si podía implementarla. A partir de esa posibilidad, se decidió crear un TDA propio para representar la ruta del desierto.

3.1. Justificación de uso

- El enunciado indica que la ruta debe modelarse mediante una lista doblemente enlazada.
- Los bandidos pueden avanzar y retroceder atravesando inicio y salida como si el recorrido fuera circular.

3.2. Cursor de lista

Para evitar que el resto del programa manipule nodos directamente, se utiliza el concepto de cursor, podría ser un nodo directamente. El cursor representa una posición válida dentro de la lista, pero el juego no necesita conocer la estructura interna del nodo.

```
typedef tNodoListaDC* tListaDC;  
typedef tNodoListaDC* tCursorDC;
```

La lista mantiene la referencia principal al último nodo, por ser circular. Sin embargo, el resto del código accede al recorrido mediante primitivas como obtenerPrimeroDC, siguienteDC, anteriorDC, avanzarNDC, retrocederNDC, verActualDC, modificarActualDC y mismoCursorDC.

3.3. Primitivas principales

Primitiva	Uso dentro del juego
obtenerPrimeroDC	Devuelve el cursor al primer casillero del tablero.
siguienteDC / anteriorDC	Permiten avanzar o retroceder un cursor una posición.
avanzarNDC / retrocederNDC	Permiten mover un cursor varias posiciones sin exponer nodos.
verActualDC	Copia el contenido del casillero apuntado por el cursor.

modificarActualDC

Modifica el dato interno del casillero mediante una acción.

mismoCursorDC

Compara dos posiciones sin comparar nodos desde afuera del TDA.

4. Representación general del tablero

El tablero agrupa la ruta circular, las referencias principales de inicio y fin, la cantidad de casilleros y la lista de bandidos activos o inactivos de la partida.

```
typedef struct
{
    tListaDC casilleros;
    tCursorDC inicio;
    tCursorDC fin;
    unsigned cantidadCasilleros;
    tLista bandidos;
} tTablero;
```

- casilleros: lista doble circular que contiene todos los casilleros del recorrido.
- inicio: cursor al Campamento Inicial.
- fin: cursor a la Ciudad Refugio.
- cantidadCasilleros: permite recorrer de forma controlada una lista circular y evitar ciclos infinitos.
- bandidos: lista dinámica de estructuras tBandido.

Aunque en una lista circular el inicio podría deducirse a partir del final, se decidió mantener inicio y fin explícitos en tTablero para simplificar validaciones y hacer más legible el código de juego.

5. Representación de casilleros

Cada posición del recorrido guarda un tCasillero. El casillero contiene el evento fijo de la posición y marcas de ocupación para jugador y bandido.

```
typedef struct
{
    tTipoEvento tipoEvento;
    int tieneJugador;
    unsigned numeroCasillero;
    unsigned idBandido;
} tCasillero;
```

5.1. Decisión sobre idBandido

Cada casillero guarda el id del bandido ubicado allí. El valor 0 representa que no hay bandido.

```
idBandido == 0 // no hay bandido
idBandido != 0 // hay un bandido con ese identificador
```

Esta decisión reduce el acoplamiento entre el tablero y la lista de bandidos. Cuando se necesita modificar un bandido, se lo busca en la lista por id mediante una primitiva genérica de lista.

6. Lista simple dinámica y primitivas agregadas

Además de las primitivas tradicionales de lista simple, se agregaron primitivas pertinentes a cada caso.

7. Configuración del juego

El archivo config.txt permite definir los parámetros iniciales de cada partida: cantidad de posiciones, vidas iniciales, cantidad máxima de bandidos, premios, vidas extra, oasis y tormentas.

7.1. Lectura de configuración

La lectura se realiza línea por línea. Cada línea se divide en clave y valor usando el carácter =. Luego se carga el valor correspondiente en la estructura tConfig.

- leerConfiguracion inicializa la estructura para evitar basura.
- trozarLinea separa clave y valor.
- cargarValor asigna el valor correspondiente según la clave leída.
- cargarConfiguracion combina lectura y validación.

7.2. Validaciones

- cantidad_posiciones debe ser al menos 3.
- vidas_inicio debe ser mayor a 0.
- La cantidad total de eventos no debe superar los casilleros disponibles.
- Los elementos hostiles se limitan con MAX_PORCENTAJE_HOSTILES.
- Los elementos favorables se limitan con MAX_PORCENTAJE_FAVORABLES.

Los casilleros disponibles se calculan restando inicio y salida:

```
casillerosDisponibles = cantidad_posiciones - 2;
```

8. Generación del tablero

La generación del tablero se realiza en etapas ordenadas para mantener simple la lógica y facilitar el control de errores.

1. Inicializar tablero.
2. Crear casilleros.
3. Colocar eventos fijos.
4. Colocar bandidos.
5. Validar que el tablero sea jugable.
6. Exportar el tablero a caravana.txt.

8.1. Creación de casilleros

Se crean tantos casilleros como indique la configuración. El primer casillero se marca como INICIO, el último como SALIDA y el resto como DESPEJADO.

8.2. Colocación de eventos

Los eventos fijos se colocan en casilleros aleatorios libres. Para que un casillero sea libre debe estar despejado, no tener jugador y no tener bandido.

La cantidad real de eventos no necesariamente coincide con el máximo establecido. Se calcula un mínimo porcentual y luego se elige aleatoriamente un valor entre ese mínimo y el máximo.

```
cantidadReal = (maximo * PORCENTAJE_MINIMO_EVENTOS) / 100;
if(cantidadReal == 0)
    cantidadReal = 1;
cantidadReal += rand() % (maximo - cantidadReal + 1);
```

Con esto se evita que el tablero quede demasiado vacío, pero se mantiene la variación entre partidas.

8.3. Colocación inicial de bandidos

Los bandidos se crean en casilleros despejados. Cada bandido se guarda en la lista dinámica de bandidos y el casillero solo almacena su idBandido.

Aunque durante la partida un bandido puede terminar sobre un casillero con evento, inicialmente se ubica solo sobre casilleros despejados para generar un escenario inicial más claro.

8.4. Validación del tablero

Luego de generar el tablero, se valida que exista al menos una casilla sin bandido dentro de las primeras posiciones alcanzables por el dado desde el inicio. Como el dado va de 1 a 6, se revisan como máximo las primeras seis posiciones posteriores al inicio.

Si el tablero no es jugable, se vacía y se genera nuevamente.

9. Jugador

El jugador mantiene su identificación, estado de partida, posición actual, cantidad de movimientos e historial.

```
typedef struct tJugador
{
    unsigned idJugador;
    char nickname[TAM_NICKNAME];
    int vidas;
    int puntaje;
    int protegidoPorOasis;
    int pierdeTurno;
    tCursorDC posicionActual;
    unsigned cantMovimientos;
    tLista historialMovimientos;
} tJugador;
```

9.1. Inicialización del estado

Al iniciar una partida se reinician vidas, puntaje, protección de oasis, pérdida de turno, contador de movimientos e historial. Además, se coloca al jugador en el casillero de inicio y se marca ese casillero mediante modificarActualDC.

9.2. Movimiento del jugador

El jugador tira un dado entre 1 y 6. Si puede retroceder, se le permite elegir dirección. Si no puede retroceder la cantidad exacta obtenida, avanza obligatoriamente.

- El jugador no puede retroceder atravesando el inicio.
- Si avanza y llega a la salida antes de consumir todos los pasos, rebota y continúa hacia atrás.
- El movimiento total consume exactamente la cantidad indicada por el dado.

9.3. Historial de movimientos

El historial del jugador se guarda en una lista dinámica independiente de la cola de movimientos del turno. El formato de muestra es F para avanzar y B para retroceder, seguido de la cantidad de casilleros.

Cuando el movimiento rebota en la salida, puede guardarse segmentado, por ejemplo F1 y B3. Esto representa el recorrido real realizado.

10. Bandidos

Los bandidos son entidades móviles almacenadas en una lista dinámica. Cada bandido conoce su id, su posición actual y si está activo.

```
typedef struct tBandido
{
    unsigned idBandido;
    tCursorDC posicionActual;
    int activo;
} tBandido;
```

10.1. Bandido activo/inactivo

Cuando un bandido intercepta al jugador, no se elimina físicamente de la lista. Se marca como inactivo. Esta decisión evita eliminar nodos durante la partida y simplifica el procesamiento de turnos.

10.2. Dirección del bandido

Para decidir hacia dónde moverse, el bandido calcula la distancia hacia adelante hasta el jugador y la distancia hacia atrás. Luego elige la dirección más corta. Si ambas distancias son iguales, se toma ADELANTE como criterio por defecto.

10.3. Cantidad de movimiento

La cantidad máxima de movimiento de un bandido se define con MAX_MOVIMIENTO_BANDIDO. La función genera una cantidad inicial aleatoria y, si ese destino no es válido, prueba otras cantidades posibles manteniendo la misma dirección.

10.4. Restricciones

- El bandido puede atravesar inicio y salida durante el recorrido.
- El bandido no puede terminar su movimiento en INICIO.
- El bandido no puede terminar su movimiento en SALIDA.
- No puede haber más de un bandido en el mismo casillero.
- Sí puede terminar sobre un casillero que tenga premio, oasis, tormenta o vida extra.

11. Ajuste de destino de bandidos

Como los movimientos se calculan antes de aplicarse, puede ocurrir que dos bandidos calculen el mismo destino usando el estado anterior del tablero. Para resolverlo se creó una función de ajuste de destino.

7. Se lee el casillero destino calculado.
8. Si no tiene bandido, el destino se mantiene.
9. Si ya tiene bandido, se avanza en la misma dirección del movimiento.
10. Se busca una casilla válida: sin bandido, que no sea inicio y que no sea salida.
11. Si no se encuentra una alternativa, el bandido queda en su origen.

Esta decisión mantiene la regla de no superponer bandidos previamente elegida por simplicidad a la hora de resolver los movimientos y hacer más abarcativo el juego en el tablero.

12. Movimientos y cola de turno

Cada turno se divide en dos momentos: preparación y procesamiento. En la preparación se generan movimientos, pero no se modifica el tablero. En el procesamiento se desencolan movimientos y se actualiza el estado real.

```
typedef struct
```

```

{
    tTipoActor actor;
    unsigned idActor;
    tCursorDC origen;
    tCursorDC destino;
    tDireccion direccion;
} tMovimiento;

```

12.1. Preparar turno

12. Si el jugador pierde turno, se saltea su movimiento y se mueven los bandidos.
13. Si no pierde turno, se espera ENTER para tirar el dado.
14. Se genera el movimiento del jugador y se encola.
15. Se recorren los bandidos mediante mapLista.
16. Por cada bandido activo se genera y encola su movimiento.

12.2. Procesar movimientos

La cola se procesa hasta quedar vacía. Si el movimiento es del jugador, se actualiza su posición, se marca el casillero destino y se aplica el evento. Si el movimiento es de un bandido, se busca por id, se ajusta el destino si corresponde y se actualiza su posición real en la lista de bandidos.

13. Eventos

Los eventos solo se aplican cuando el jugador cae en el casillero destino. Las posiciones intermedias no producen efectos.

Evento	Efecto
PREMIO	Suma 1 punto y el evento se consume.
VIDA_EXTRA	Suma 1 vida y el evento se consume.
OASIS	Activa protección contra el próximo peligro y se consume.
TORMENTA	Si no hay protección, el jugador pierde el próximo turno. Si hay protección, se consume y no produce efecto.
SALIDA	La partida se considera ganada cuando el jugador queda en esa posición.

14. Colisión con bandidos

Luego de procesar los movimientos y aplicar eventos, se verifica si en la posición actual del jugador hay un bandido. Esto se hace leyendo el casillero y evaluando idBandido.

- Si el jugador no está protegido, pierde una vida y vuelve al inicio.
- Si el jugador está protegido por oasis, no pierde vida y la protección se consume.
- En ambos casos, el bandido que interceptó al jugador se desactiva.
- El casillero queda sin bandido, asignando idBandido = 0.

Cabe destacar que elegimos validar los eventos primero siempre, como para aprovechar los beneficios de los mismos, en caso de haber sido interceptado por un bandido.

15. Finalización de partida y puntaje mínimo por victoria

Al finalizar la partida se muestra el resultado, se muestra el historial de movimientos y se guarda un registro en partidas.dat.

Se agregó una regla de diseño: si el jugador gana la partida, se le suma 1 punto. Esto equilibra el hecho de que los jugadores podrían conformarse simplemente agarrando premios, sin ganar la partida, ya que de todas formas los puntos se le serán sumados.

Esta regla se aplica en finalizarPartida antes de guardar el registro, para que el ranking tome el puntaje definitivo.

16. Archivo de partidas

Las partidas se guardan en el archivo binario partidas.dat. Cada registro contiene id de partida, id de jugador, puntaje, cantidad de movimientos y resultado.

```
typedef struct
{
    unsigned idPartida;
    unsigned idJugador;
    unsigned puntaje;
    unsigned cantMovimientos;
    tResultadoPartida resultado;
} tPartida;
```

El id de partida se calcula leyendo el último id existente y sumando uno. La cantidad de movimientos se toma desde el contador del jugador, no desde la cola, porque la cola se vacía al terminar cada turno.

17. Gestión de jugadores e índice

El ingreso del jugador se realiza por nombre. Como el nombre real no es clave única, el sistema busca todos los jugadores que tengan ese nombre y muestra una lista de nicknames asociados.

17. El usuario ingresa su nombre.
18. Se recorre jugadores.dat buscando registros con ese nombre.
19. Se arma una lista dinámica con los nicknames encontrados.
20. El usuario elige si alguno de esos nicknames le pertenece.
21. Si elige uno existente, se busca ese nickname en el índice de jugadores.
22. Si no existe o no corresponde, se genera un nuevo nickname y se da de alta el jugador.

17.1. Índice por nickname

El índice principal de jugadores está implementado con un árbol binario de búsqueda. La clave es el nickname, porque permite identificar un jugador de forma única dentro del sistema.

```
typedef struct
{
    char nickname[TAM_NICKNAME];
    unsigned posicionRegistro;
} tIndiceJugador;
```

La posición almacenada permite acceder directamente a jugadores.dat mediante fseek. En la implementación actual se usa la posición lógica de registro.

17.2. Regeneración y persistencia

Al iniciar el programa se intenta cargar el índice desde `indiceJugadores.idx`. Si no existe, se regenera a partir de `jugadores.dat` y luego se carga en el árbol. Al cerrar o luego de modificar jugadores, el índice se guarda nuevamente en archivo binario.

El archivo de índice se persiste ordenado, y luego se carga mediante una estrategia de árbol balanceado desde archivo ordenado, lo que permite obtener una estructura de búsqueda más eficiente que insertar registros en el orden del archivo.

18. Ranking

Se calcula a partir de `partidas.dat`, ya que ese archivo es la fuente de verdad para los puntajes obtenidos.

23. Se abre `partidas.dat`.
24. Se lee cada partida.
25. Se agrupa por `idJugador` en una lista dinámica.
26. Si un jugador aparece más de una vez, se acumula su puntaje.
27. Se ordena por puntaje descendente.
28. Para mostrar el nickname, se accede a `jugadores.dat` usando el `idJugador`.

La estructura usada para agrupar ranking es:

```
typedef struct
{
    unsigned idJugador;
    char nickname[TAM_NICKNAME];
    unsigned total;
} tRanking;
```

Se eligió lista dinámica porque no se conoce de antemano cuántos jugadores tendrán partidas registradas.

19. Ayuda y archivos de texto

El menú incluye una opción de ayuda. La ayuda se muestra leyendo un archivo de texto. Si el archivo no existe, el programa puede generarlo con una explicación breve de reglas, símbolos y objetivo del juego.

- `ARCH_AYUDA`: archivo de ayuda.
- `ARCH_CARAVANA`: archivo de texto con el tablero generado.
- `ARCH_CONFIGURACION`: archivo de configuración inicial.

20. Utilidades generales

El módulo `Utils` concentra funciones auxiliares de uso general.

- `tirarDado`: genera un número entre 1 y 6. Se evita sesgo usando un límite basado en `RAND_MAX`.
- `abrirArchivo`: centraliza apertura de archivos y devuelve códigos de error del proyecto.
- `trozarLinea`: separa clave y valor en la lectura de configuración.
- `pausar`: pausa la consola esperando ENTER.

21. Flujo general del programa

29. Se inicializa la semilla aleatoria con `srand(time(NULL))`.
30. Se crea el árbol índice de jugadores.
31. Se carga o regenera el índice de jugadores.
32. Se ingresa o da de alta el jugador.
33. Se guarda el índice actualizado.
34. Se muestra el menú principal.

35. Desde el menú se puede jugar, ver ranking, consultar ayuda o salir.
36. Al salir se libera el árbol índice.

.